PROJECT DOCUMENTATION

Automated Network Request Management in ServiceNow

1. Introduction

Project Title: Automated Network Request Management in ServiceNow

Team Members:

- Team Leader: [Enter Name]
- Team Member 1: [Enter Name]
- Team Member 2: [Enter Name]
- Team Member 3: [Enter Name]

Roles and Responsibilities:

- Project planning and requirement analysis.
- ServiceNow form and table configuration.
- Workflow automation and request assignment.
- Approval and notification configuration.
- Testing, documentation, and deployment support.

Note: Update the team members and roles according to your actual project team.

2. Project Overview

Purpose

The purpose of the Automated Network Request Management in ServiceNow project is to simplify and automate the process of submitting, assigning, tracking, and resolving network-related requests within an organization.

The system provides a centralized platform where employees can submit requests for network connectivity, VPN access, and other network-related support services. Based on configured rules, requests can be assigned to appropriate support teams and routed through approval workflows when required.

The project aims to reduce manual effort, improve request visibility, and make network support operations more organized and efficient.

Features

The main features of the project are:

1. **Network Request Submission:** Allows users to submit network requests through a configured request form.
2. **Request Validation:** Validates mandatory fields before submission.
3. **Unique Request Number:** Identifies each request with a unique number.
4. **Automated Assignment:** Routes requests to the appropriate support team using configured assignment rules.
5. **Approval Workflow:** Supports approval or rejection of requests that require authorization.
6. **Email Notifications:** Notifies relevant users about configured request events.
7. **Request Tracking:** Allows users and authorized support personnel to track request status.
8. **SLA Management:** Supports monitoring of response and resolution targets.
9. **Reports and Dashboards:** Provides request statistics for monitoring and analysis.
10. **Request Resolution:** Allows support personnel to record work notes and resolution details.

3. Architecture

Platform Architecture

The application is designed around the ServiceNow platform. Unlike a traditional MERN-stack application, it can use ServiceNow's built-in user interface, data tables, workflow automation, access controls, and reporting capabilities.

Frontend

The frontend is the user-facing interface through which employees submit network requests and view their status.

Technologies and components:

- ServiceNow Service Catalog or Record Producers, depending on the implementation.
- ServiceNow forms and record lists.
- Client Scripts for client-side form behavior, if required.

- UI Policies for controlling field visibility and behavior, if configured.
- Service Portal or Employee Center, if used in the project.

The frontend collects request details and displays information according to the user's permissions.

Backend

The backend logic manages request processing, assignment, approvals, notifications, and status updates.

Technologies and components:

- Flow Designer for workflow automation.
- Business Rules for server-side logic, if required.
- Script Includes for reusable server-side logic, if required.
- Assignment rules or configured routing logic.
- Approval and notification mechanisms.
- ServiceNow platform APIs, where required.

The backend processes submitted information and applies the business rules configured for network request management.

Database

ServiceNow provides a platform-managed relational data layer through its tables and records.

The project may use the following tables, depending on its implementation:

Table	Purpose
`sc_request`	Stores request-level information for Service Catalog requests
`sc_req_item`	Stores requested catalog items
`sc_task`	Stores catalog fulfillment tasks
`sys_user`	Stores user information
`sys_user_group`	Stores support group information
`sysapproval_approver`	Stores approval records for applicable workflows

Custom network request table	Stores network request data if a custom table is implemented

The exact tables depend on whether the project uses Service Catalog items, record producers, custom tables, or another request structure.

Data Interaction

1. A user submits a network request.
2. ServiceNow validates the submitted information.
3. The request is stored in the configured table or catalog request records.
4. Workflow rules determine assignment and approval requirements.
5. Request status and work details are updated during processing.
6. Reports retrieve relevant records to display request statistics.

4. Setup Instructions

Prerequisites

The following are required to develop, configure, or test the project:

- Access to a ServiceNow development or personal developer instance.
- A supported web browser, such as Google Chrome or Microsoft Edge.
- Appropriate permissions to create and configure application components.
- Basic knowledge of ServiceNow tables, forms, workflows, and access controls.
- ServiceNow developer documentation for reference.
- Git, if the project uses version control for exported scripts or documentation.

Node.js, Express.js, React, and MongoDB are not mandatory for a ServiceNow-native implementation.

Installation and Configuration

Step 1: Obtain ServiceNow Instance Access

Create or access an authorized ServiceNow development instance.

Step 2: Sign In

Log in using the authorized account and open the relevant application or development area.

Step 3: Configure Request Management

Create or configure the required request form, catalog item, record producer, or custom table based on the project design.

Step 4: Configure Fields

Add the necessary fields, such as:

- Request category.
- Short description.
- Detailed description.
- Requested-for user.
- Priority, if applicable.
- Assignment group.
- Approval status.
- Request status.
- Resolution details.

Step 5: Configure Automation

Set up the required Flow Designer flows, assignment rules, approval activities, notifications, or server-side scripts.

Step 6: Configure Security

Apply appropriate roles, access controls, and permissions so users can access only the information and actions authorized for them.

Step 7: Test the Application

Submit sample requests and verify validation, assignment, approvals, notifications, and status updates.

Step 8: Verify Reports

Check that reports and dashboards display the correct information for the configured request records.

Note: These steps describe a typical configuration process. The exact menus and components depend on the ServiceNow release and application design.

5. Folder Structure

A ServiceNow-native application does not necessarily use a traditional client/server folder structure. Application components are typically managed within the ServiceNow platform.

If exported configuration files, scripts, diagrams, and documentation are maintained in GitHub, the repository can be organized as follows:

```
Automated-Network-Request-Management/
|
|-- README.md
|
|-- documentation/
|   |-- project-overview.md
|   |-- architecture.md
|   |-- api-documentation.md
|   |-- test-cases.md
|   |-- screenshots/
|
|-- scripts/
|   |-- business-rules/
|   |-- client-scripts/
|   |-- script-includes/
|
|-- workflows/
|   |-- flow-descriptions.md
|   |-- assignment-rules.md
|   |-- approval-workflows.md
|
|-- reports/
|   |-- dashboard-design.md
|
|-- test-data/
|   |-- sample-network-requests.csv
|
|-- diagrams/
|   |-- data-flow-diagram.png
|   |-- solution-architecture.png
|
|-- .gitignore
```

This is a suggested documentation repository structure, not a claim about the actual files in the ServiceNow instance. Include only directories and files that are relevant to your implementation.

6. Running the Application

The application runs within the configured ServiceNow instance rather than requiring separate React and Node.js development servers.

Steps to Access the Application

1. Open the ServiceNow instance URL in a web browser.
2. Sign in using an authorized account.
3. Navigate to the configured network request form or service catalog.
4. Submit a sample network request.
5. Verify request creation and assignment.
6. Check the approval process if approval is required.
7. Verify notifications and request status updates.
8. Open the configured reports or dashboards.

Local Commands: Commands such as `npm start` are not required for a ServiceNow-native implementation. They are applicable only if the project also contains a separate Node.js or frontend application.

7. API Documentation

ServiceNow provides platform APIs, including Table API and other APIs, when enabled and authorized. A project may also expose custom Scripted REST APIs if required.

The following are possible API examples, not verified endpoints implemented by this project.

7.1 Retrieve Network Request Records

Method: GET

Example endpoint:

https://<instance>.service-now.com/api/now/table/<request_table>

Purpose: Retrieves records from the specified table, subject to permissions and API configuration.

Example query parameters:

?sysparm_limit=10
&sysparm_fields=number,state,short_description

Example response:

```
{
  "result": [
    {
      "number": "REQ100001",
      "state": "Open",
      "short_description": "VPN access request"
    }
  ]
}
```


The example response is illustrative. Actual field names and values depend on the configured table and API response.

7.2 Retrieve a Specific Request

Method: GET

Example endpoint:

https://<instance>.service-now.com/api/now/table/<request_table>/<sys_id>

Purpose: Retrieves a specific record using its `sys_id`, subject to access permissions.

7.3 Create a Network Request Record

Method: POST

Example endpoint:

https://<instance>.service-now.com/api/now/table/<request_table>

Example request body:


```
{
  "short_description": "Network connectivity issue",
  "description": "Unable to connect to the office network",
  "category": "network"
}
```


Purpose: Creates a record when the selected table, fields, permissions, and API configuration support the operation.

7.4 Update a Network Request

Method: PATCH

Example endpoint:

https://<instance>.service-now.com/api/now/table/<request_table>/<sys_id>

Example request body:

```
{
  "work_notes": "Network connectivity has been checked"
}
```

Purpose: Updates permitted fields on an existing record.

Important: Replace `<instance>` and `<request_table>` with the actual values. Confirm the table, field names, authentication method, ACL permissions, and API availability in your ServiceNow instance. If the project does not expose APIs, document this section as “Not applicable” rather than claiming these endpoints were implemented.

8. Authentication and Authorization

The project relies on ServiceNow authentication and authorization mechanisms.

Authentication

Authentication verifies the identity of a user before granting access to the ServiceNow instance. Depending on the organization's configuration, this may use local accounts, single sign-on (SSO), or another supported authentication method.

Authorization

Authorization determines which actions and records a user can access.

Role-based access control and Access Control Lists (ACLs) can restrict access to request forms, records, fields, and administrative functions.

Typical roles may include:

- **Requester:** Submits requests and views permitted request information.
- **Network Support Agent:** Handles assigned requests and updates work details.
- **Approver:** Approves or rejects requests requiring authorization.
- **Administrator:** Configures the application and manages authorized administrative tasks.

These are example role definitions. Actual access depends on the roles, groups, and ACLs configured in the instance.

API Security

If APIs are used, requests should use an approved authentication mechanism, such as OAuth or authorized platform authentication. Access must be restricted through permissions and ACLs.

Credentials, access tokens, and confidential instance information must not be stored in public repositories or shared in screenshots.

9. User Interface

The user interface provides the main interaction points for requesters, support teams, managers, and administrators.

The following screenshots should be captured from the actual ServiceNow implementation:

1. **Network Request Form:** Shows the fields used to submit a request.
2. **Request Record:** Shows a submitted request and its unique identifier.
3. **Assignment Details:** Shows the assigned support group or agent.
4. **Approval Interface:** Shows approval status and available actions.
5. **Request Tracking:** Shows the current state of a request.
6. **Notification Example:** Shows an actual configured notification, if available.
7. **Reports and Dashboard:** Shows request statistics and status summaries.

Each screenshot should include a figure number, a descriptive caption, and a short explanation.

10. Testing

Testing verifies that the application meets its functional and operational requirements.

Functional Testing

Test ID	Test Scenario	Expected Outcome
TC01	Submit a request with valid details	Request is created
TC02	Submit a request with missing mandatory information	Validation prevents incomplete submission

TC03	Submit a new request	A request identifier is generated
TC04	Submit a request matching an assignment rule	Correct configured assignment is applied
TC05	Submit a request requiring approval	Approval process is triggered
TC06	Approve or reject a request	Approval outcome is recorded
TC07	Update request status	Updated status is displayed
TC08	Trigger a notification event	Configured notification is generated
TC09	Record request resolution	Resolution information is saved
TC10	Open a report	Authorized request data is displayed

Performance and Security Testing

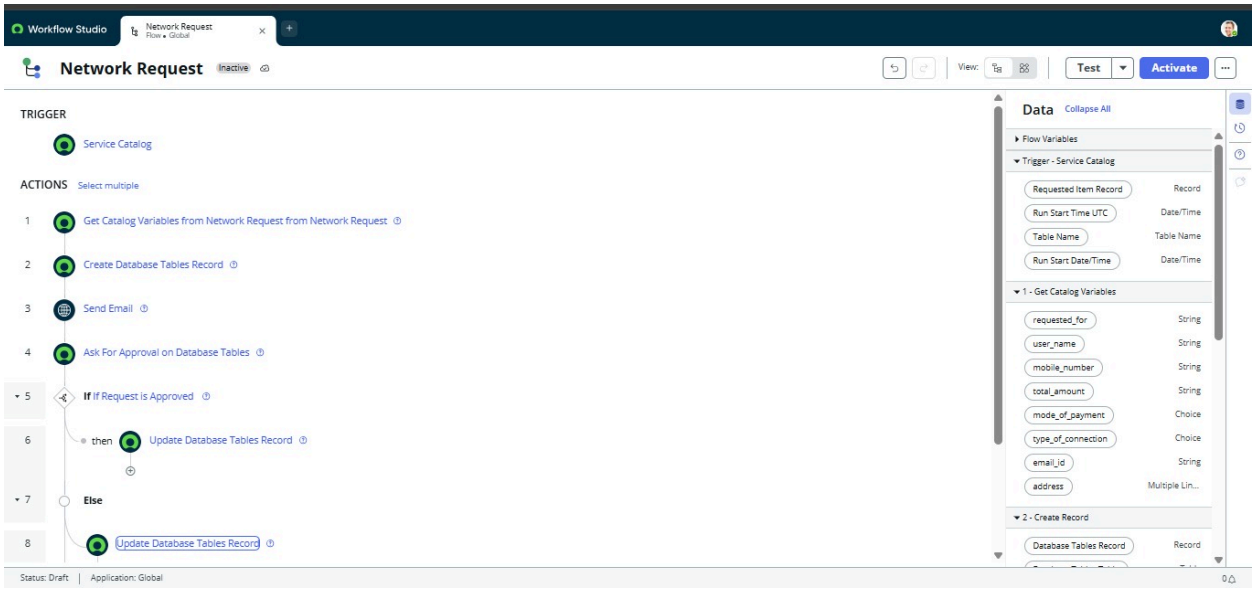
- Check response times for request submission and record retrieval.
- Evaluate behavior with multiple authorized users.
- Verify workflow execution under representative conditions.
- Check report loading times.
- Confirm that users cannot access unauthorized records or actions.
- Verify that mandatory fields and approval requirements are enforced.
- Review application logs for unexpected errors.

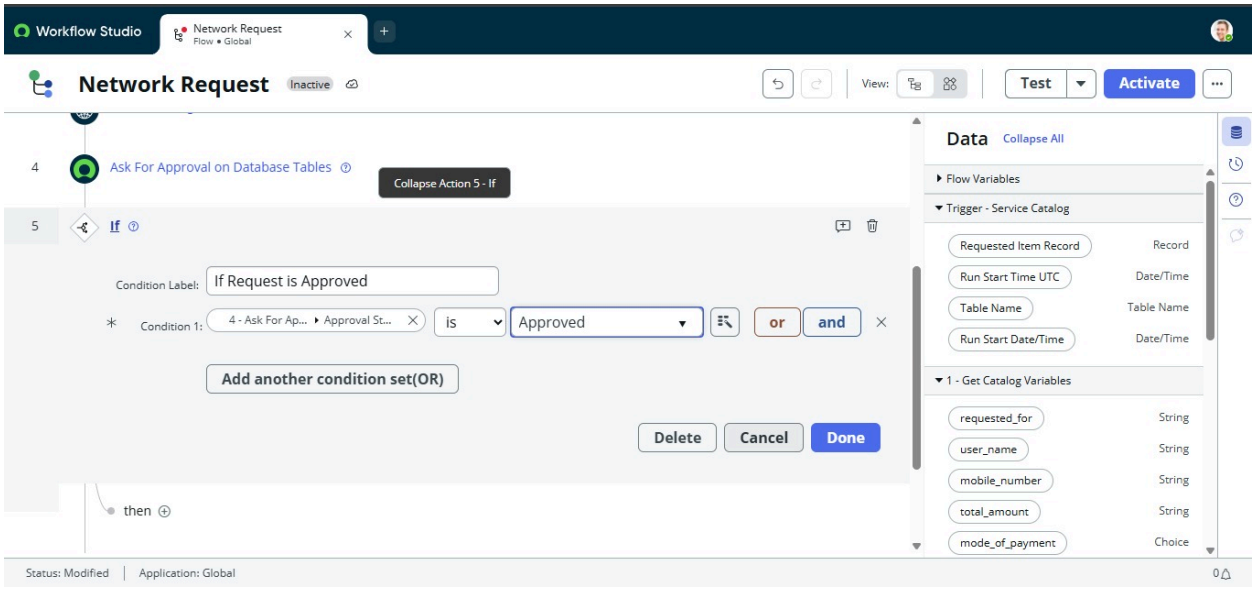
Testing Tools

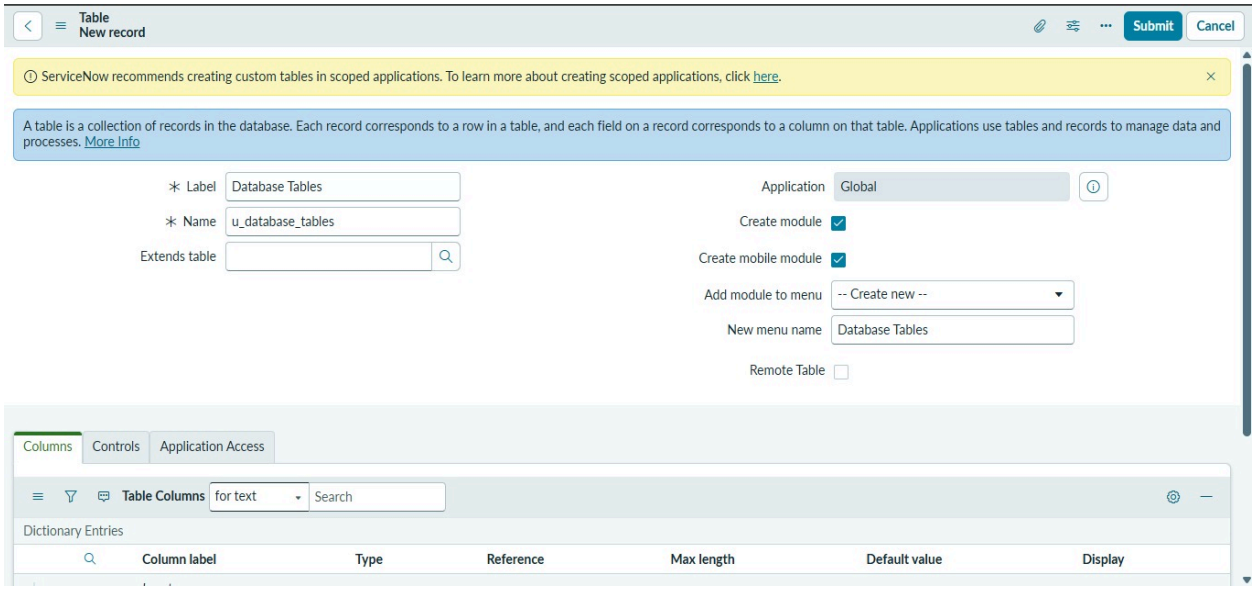
Potential tools include ServiceNow's built-in testing and debugging capabilities, ATF (Automated Test Framework) where available, browser developer tools, and authorized API clients such as Postman when API testing is required.

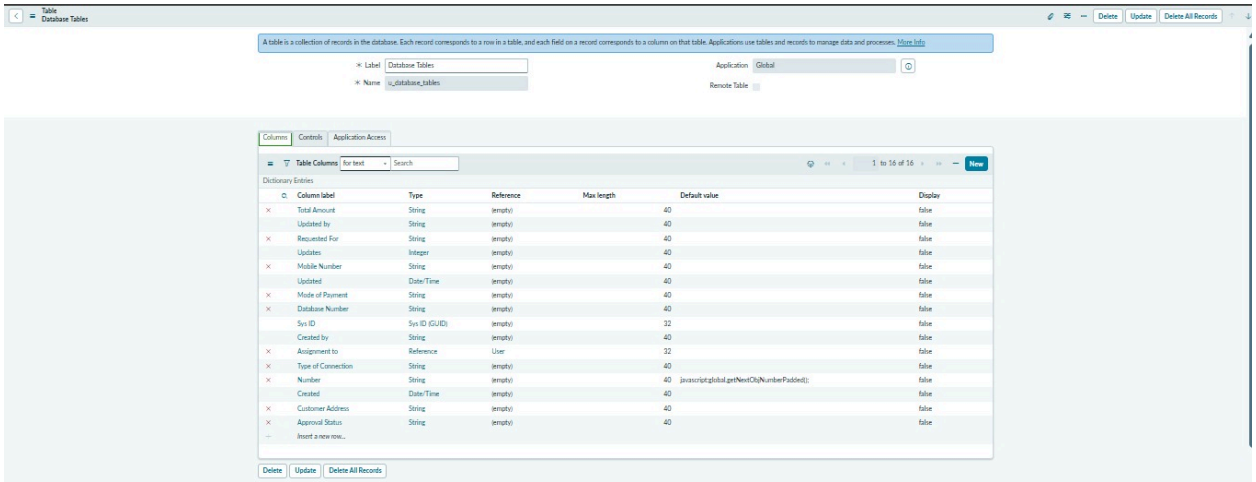
Actual test results should be recorded as Pass, Fail, or Blocked, with observations and corrective actions.

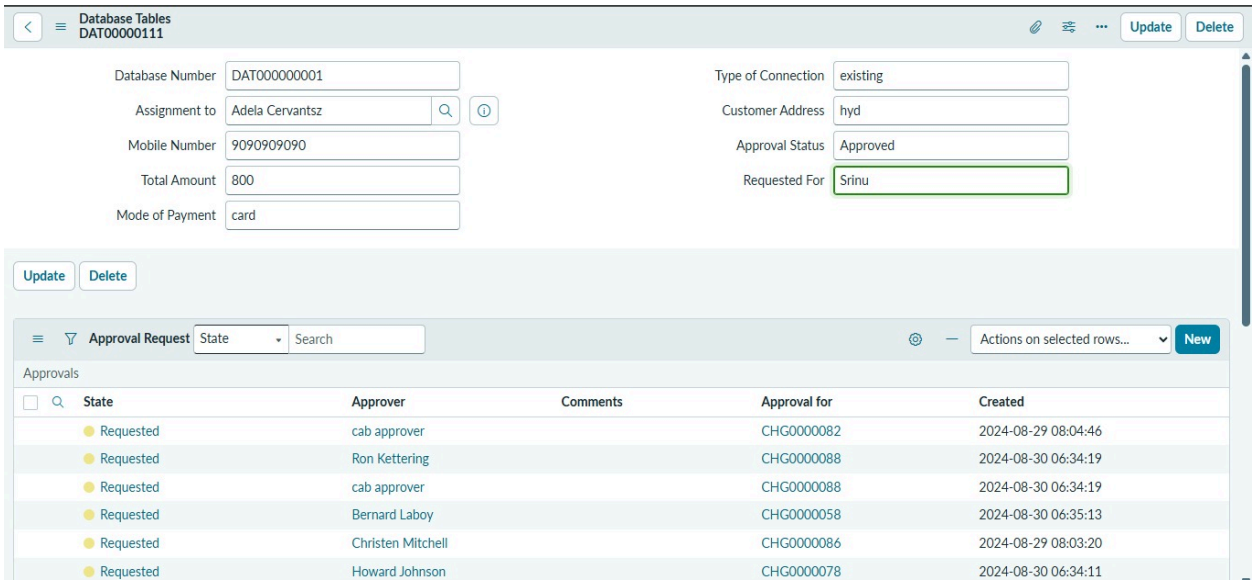
11. Screenshots or Demo

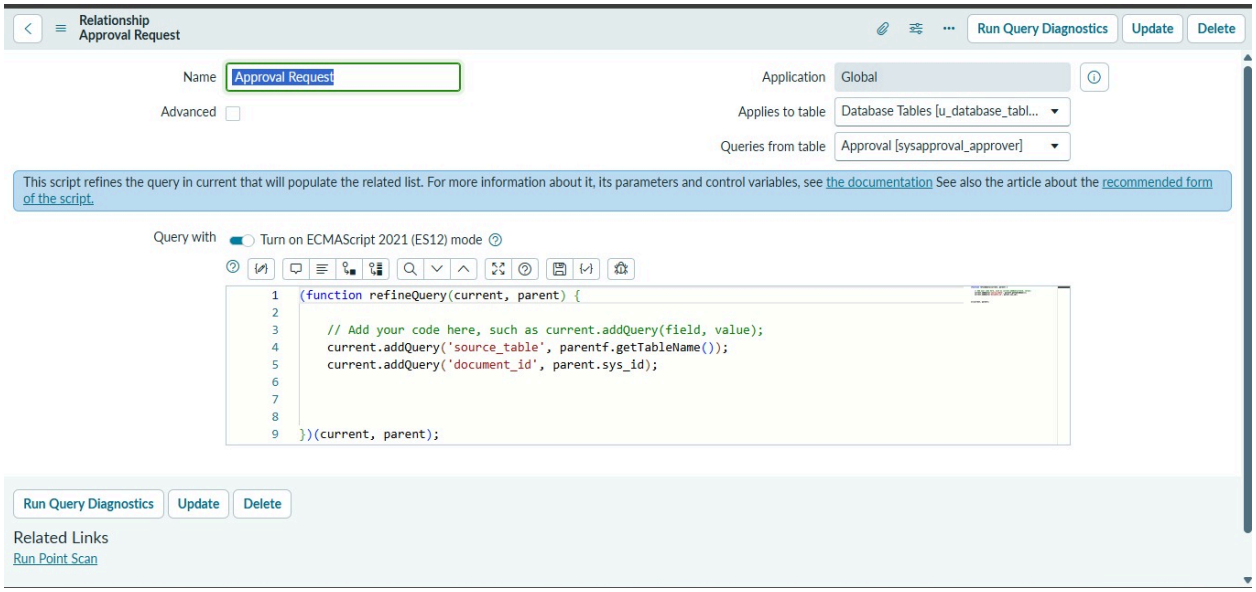

Workflow Studio
Network Request
Flow • Global
Network Request
Inactive
View:
Test
Activate
TRIGGER
Service Catalog
ACTIONS
Select multiple
1 Get Catalog Variables from Network Request from Network Request
2 Create Database Tables Record
3 Send Email
4 Ask For Approval on Database Tables
5 If If Request is Approved
6 then Update Database Tables Record
7 Else
8 Update Database Tables Record
Data
Collapse All
Flow Variables
Trigger - Service Catalog
Requested Item Record Record
Run Start Time UTC Date/Time
Table Name Table Name
Run Start Date/Time Date/Time
1 - Get Catalog Variables
requested_for String
user_name String
mobile_number String
total_amount String
mode_of_payment Choice
type_of_connection Choice
email_id String
address Multiple Lin...
2 - Create Record
Database Tables Record Record
Status: Draft
Application: Global

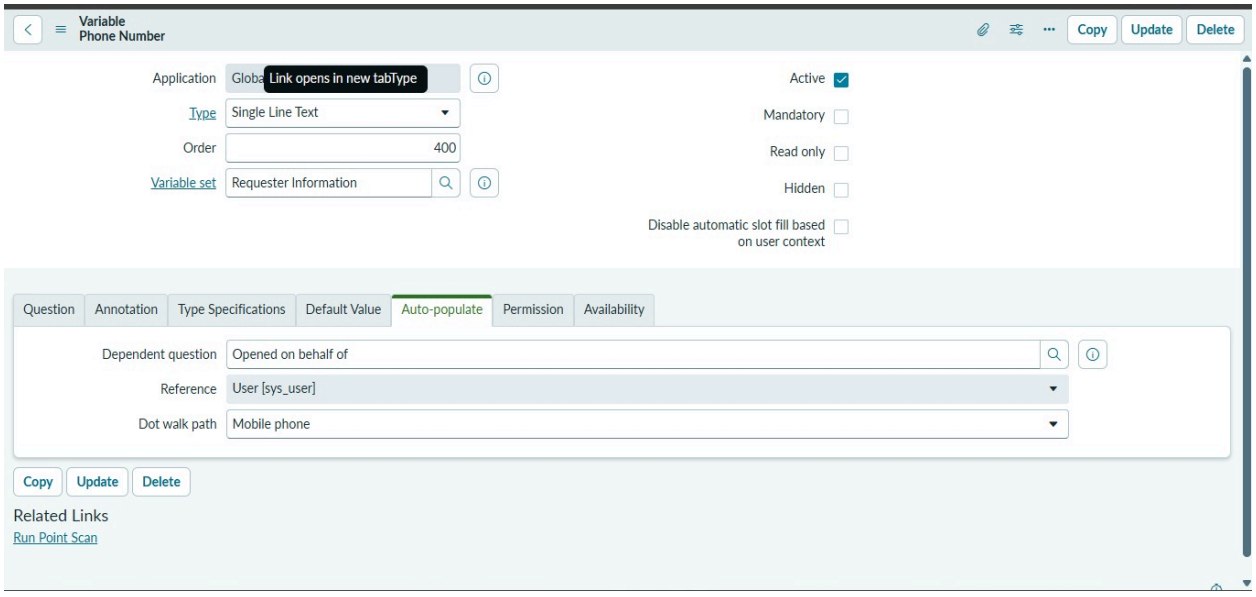

Workflow Studio
Network Request
Flow • Global
Network Request
Inactive
View:
Test
Activate
4 Ask For Approval on Database Tables
Collapse Action 5 - If
5 If
Condition Label: If Request is Approved
Condition 1: 4 - Ask For Ap... Approval St... is Approved or and
Add another condition set(OR)
Delete
Cancel
Done
then
Data
Collapse All
Flow Variables
Trigger - Service Catalog
Requested Item Record Record
Run Start Time UTC Date/Time
Table Name Table Name
Run Start Date/Time Date/Time
1 - Get Catalog Variables
requested_for String
user_name String
mobile_number String
total_amount String
mode_of_payment Choice
Status: Modified
Application: Global

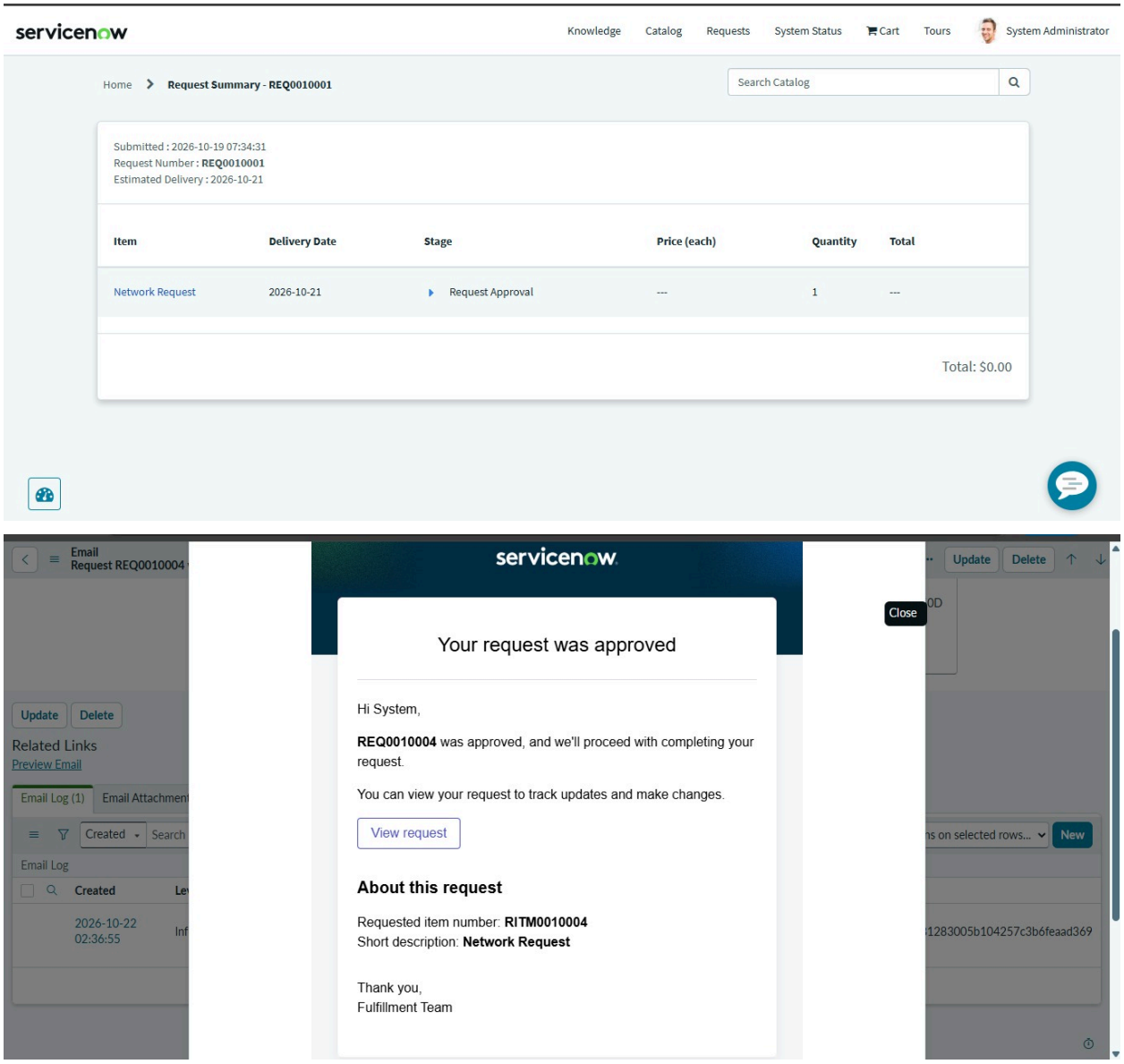

Table
New record
Submit
Cancel
ServiceNow recommends creating custom tables in scoped applications. To learn more about creating scoped applications, click here.
A table is a collection of records in the database. Each record corresponds to a row in a table, and each field on a record corresponds to a column on that table. Applications use tables and records to manage data and processes. More Info
Label
Database Tables
Name
u_database_tables
Extends table
Application
Global
Create module
Create mobile module
Add module to menu
-- Create new --
New menu name
Database Tables
Remote Table
Columns
Controls
Application Access
Table Columns
for text
Search
Dictionary Entries
Column label
Type
Reference
Max length
Default value
Display




Database Tables
DAT00000111
Update
Delete
Database Number
DAT000000001
Assignment to
Adela Cervantsz
Mobile Number
9090909090
Total Amount
800
Mode of Payment
card
Type of Connection
existing
Customer Address
hyd
Approval Status
Approved
Requested For
Srinu
Update
Delete
Approval Request
State
Search
Actions on selected rows...
New
Approvals
State
Approver
Comments
Approval for
Created
Requested
cab approver
CHG0000082
2024-08-29 08:04:46
Requested
Ron Kettering
CHG0000088
2024-08-30 06:34:19
Requested
cab approver
CHG0000088
2024-08-30 06:34:19
Requested
Bernard Laboy
CHG0000058
2024-08-30 06:35:13
Requested
Christen Mitchell
CHG0000086
2024-08-29 08:03:20
Requested
Howard Johnson
CHG0000078
2024-08-30 06:34:11


Relationship
Approval Request
Run Query Diagnostics
Update
Delete
Name
Approval Request
Advanced
Application
Global
Applies to table
Database Tables [u_database_tabl...
Queries from table
Approval [sysapproval_approver]
This script refines the query in current that will populate the related list. For more information about it, its parameters and control variables, see the documentation See also the article about the recommended form of the script.
Query with
Turn on ECMAScript 2021 (ES12) mode
1 (function refineQuery(current, parent) {
2
3 // Add your code here, such as current.addQuery(field, value);
4 current.addQuery('source_table', parentf.getTableName());
5 current.addQuery('document_id', parent.sys_id);
6
7
8
9 })(current, parent);
Run Query Diagnostics
Update
Delete
Related Links
Run Point Scan


Variable
Phone Number
Copy
Update
Delete
Application
Global
Link opens in new tab
Type
Single Line Text
Order
400
Variable set
Requester Information
Active
Mandatory
Read only
Hidden
Disable automatic slot fill based on user context
Question
Annotation
Type Specifications
Default Value
Auto-populate
Permission
Availability
Dependent question
Opened on behalf of
Reference
User [sys_user]
Dot walk path
Mobile phone
Copy
Update
Delete
Related Links
Run Point Scan



12. Known Issues

Known issues should be documented based on actual testing.

Potential issues to investigate include:

- Incorrect assignment if routing conditions are misconfigured.
- Delayed notifications due to email configuration or processing.
- Approval delays when approvers are unavailable.
- Incomplete request records if mandatory-field validation is insufficient.
- Incorrect report results if filters or data conditions are configured incorrectly.

- Permission errors when roles or ACLs are not configured appropriately.
- API failures due to incorrect authentication, field names, or access permissions.

These are potential issues to check, not confirmed defects in the project. Replace this list with the issues observed during actual testing.

13. Future Enhancements

The project can be enhanced through the following improvements:

1. **AI-Based Request Classification:** Automatically categorize requests based on their descriptions.
2. **Predictive SLA Monitoring:** Identify requests that may exceed their resolution targets.
3. **Network Monitoring Integration:** Generate or update requests based on authorized monitoring alerts.
4. **Knowledge Base Integration:** Recommend troubleshooting articles to users.
5. **Advanced Analytics:** Identify recurring issues and analyze support team workload.
6. **Automated Remediation:** Integrate approved scripts or automation for eligible network operations.
7. **Mobile Accessibility:** Improve request submission and tracking on mobile devices.
8. **Enhanced Security Monitoring:** Improve auditing and monitoring of sensitive network requests.
9. **Additional Integrations:** Integrate with approved IT operations and asset management systems.

Final Summary

Automated Network Request Management in ServiceNow provides a structured approach to handling network-related service requests. The solution uses platform capabilities to support request submission, assignment, approval, tracking, and reporting. Proper configuration, access control, and testing are essential to ensure reliable operation.